



Product Requirements Document (PRD)

프로젝트명: 마이 헬스 로그 API (My Health Log API)

1. 개요 (Overview)

- 제품 이름: 마이 헬스 로그 API (My Health Log API)
- 목적: 사용자가 입력한 생체 수치(체중, 키, 혈압, 혈당 등)를 기록·관리하고, 서버 측에서 신체 상태(BMI, 혈압/혈당 위험군 등)를 자동 판정 및 분석하여 제공하는 RESTful 백엔드 서비스
- 개발 환경 및 기한: 4일 (개인 프로젝트)
- 핵심 가치:
 - 신체 상태 자동 판정 알고리즘 제공
 - 파일 시스템 기반 데이터 영속성 유지
 - Docker 컨테이너를 통한 플랫폼 독립적 실행 환경 제공

2. 배경 및 목표 (Background & Goals)

2.1 문제 정의 (Problem Statement)

- 수동 건강 기록 관리 시 BMI 계산, 혈압·혈당 단계 판정 등 수치 해석의 번거로움 존재
- 클라이언트 애플리케이션 개발 시 재사용 가능한 표준 백엔드 API 서비스 필요

2.2 목표 (Goals)

- 기능적 목표: CRUD 기능, 날짜 범위 검색, 데이터 기반 통계 수치 자동 산출 기능 완성
- 기술적 목표: FastAPI 프레임워크 활용, Pydantic을 통한 데이터 유효성 검증, JSON 기반 영속성 구현, Docker 컨테이너화

3. 사용자 및 주요 유즈케이스 (User & Use Cases)

3.1 타겟 사용자

- 일별 건강 데이터를 기록하고 추적하고자 하는 일반 사용자
- 헬스케어 관련 클라이언트(웹/앱) 프론트엔드 개발자

3.2 주요 유즈케이스 (Use Cases)

1. 일별 기록 등록: 키, 몸무게, 혈압, 공복 혈당, 걸음 수, 수면 시간 등을 입력하여 등록
2. 자동 상태 진단: 등록 즉시 BMI 수치, 혈압/혈당 분류 단계 및 경고 문구 확인
3. 이력 조회 및 검색: 전체 이력 및 특정 날짜 범위(start, end) 내 기록 조회
4. 통계 확인: 누적 기록 데이터를 기반으로 한 평균 체중, 평균 혈당, 평균 혈압 조회

4. 기능 요구사항 (Functional Requirements)

4.1 REST API 엔드포인트 명세

메서드	경로	설명	비고
POST	/records	건강 기록 추가	BMI, 혈압/혈당 분류, 경고 문구 자동 계산
GET	/records	전체 기록 목록 조회	총 개수 및 전체 데이터 배열 반환
GET	/records/{id}	특정 기록 단건 조회	존재하지 않을 경우 404 Exception 반환
PUT	/records/{id}	특정 기록 수정	수정 수치 기준 재계산 수행
DELETE	/records/{id}	특정 기록 삭제	데이터 파일에서 제거
GET	/search	날짜 범위 조회	Query Parameter: start, end
GET	/stats	건강 수치 통계	평균 체중, 평균 혈당, 평균 혈압 제공

4.2 건강 데이터 분류 및 산출 로직

1. BMI 산출 Formula:

$$BMI = \frac{\text{weight (kg)}}{\left(\frac{\text{height (cm)}}{100}\right)^2}$$

2. BMI 분류 기준:

- $BMI < 18.5$: 저체중
- $18.5 \leq BMI \leq 22.9$: 정상

- $23.0 \leq BMI \leq 24.9$: 과체중
 - $BMI \geq 25.0$: 비만 (경고 메시지 생성)
3. 혈압 분류 기준:
- 수축기 < 120 mmHg 및 이완기 < 80 mmHg: 정상
 - 수축기 ≥ 140 mmHg 또는 이완기 ≥ 90 mmHg: 고혈압 (경고 메시지 생성)
 - 그 외: 주의
4. 공복 혈당 분류 기준:
- < 100 mg/dL: 정상
 - $100 \sim 125$ mg/dL: 공복혈당장애
 - ≥ 126 mg/dL: 당뇨 의심 (경고 메시지 생성)

4.3 데이터 영속성 (Data Persistence)

- 모든 데이터는 data.json 파일에 저장 및 로드
- 애플리케이션 서버가 재시작되어도 기존 기록 유지

5. 비기능 요구사항 (Non-Functional Requirements)

- 데이터 검증: Pydantic Schema를 이용하여 모든 Request Body 및 Query Parameter 타입 검증
- 문서화: OpenAPI / Swagger UI (/docs)를 통한 대화형 API 명세 자동 제공
- 컨테이너화: Dockerfile 기반 빌드 및 docker run을 통한 표준 실행 환경 보장
- 형상 관리: Git/GitHub 연동, .gitignore 및 .dockerignore를 통한 불필요 파일(venv/, data.json 등) 추적 배제

6. 기술 스택 (Tech Stack)

- **Language:** Python 3.10
- **Framework:** FastAPI
- **Data Validation:** Pydantic
- **Web Server:** Uvicorn
- **Container Environment:** Docker
- **VCS:** Git & GitHub

7. 개발 마일스톤 (Milestones)

- **Day 1:** 개발 환경 설정, Pydantic 모델 정의, 기본 CRUD 엔드포인트 구현, GitHub 초기 푸시
- **Day 2:** BMI/혈압/혈당 판정 로직 및 warnings 경고 시스템 구축, PUT 수정 기능 개발
- **Day 3:** JSON 파일 영속성 연동, 기간 검색(GET /search) 및 통계(GET /stats) 엔드포인트 완성
- **Day 4:** Dockerfile 및 requirements.txt 작성, Docker 빌드/실행 검증, README.md 완성 및

최종 제출